

05-Transport Layer

网络层及以下：主机间的通信

TCP UDP
NAT/PAT

1. ~~第四层传输层~~主要是实现了主机之间的通信。
2. 数据通信是服务于主机上的进程(Session)。

1. 第四层综述 → 进程间的通信

1. 第4层执行多项功能：

1. 分割上层应用程序数据(新的数据单元-数据段)
2. 建立端到端(end to end)的通信 (进程到进程) 只检验报头
3. 从一个终端主机向另一个终端主机发送段(第三层和第二层不进行可靠性检验，第四层完成可靠性检验，接受方认为数据错误，在第四层进行要求重传) ①
- ② 流量控制和可靠性 flow control
 1. 可以比喻为与外国人交谈：通常，您会要求外国人重复他/她的话（可靠性）并慢声说话（流量控制）
 2. 双方主机的网络的处理能力不同，缓存能力不同

2. 两个特别重要的第4层协议：

1. 传输控制协议（TCP, Transmission Control Protocol）
2. 用户数据报协议（UDP, User Datagram Protocol）

3. 将传出邮件分成多个部分,在目标站重新组合消息

4. TCP: 可靠(效率比较低，早期网络应用少，需要可靠性)

- ① 面向连接，使用确认机制，提供流量控制
- ② 软件检查 软件校验方案，校验 segment 数据段 acknowledgement { 收到：对
未：重传
3. 重新发送丢失或错误的任何内容 ② flow control

5. UDP: 不可靠 传输

1. 无连接，不使用确认，不进行流量控制
2. 不提供用于细分的软件检查
3. 直接丢弃错误的报文，而不进行其他操作。 无流控制

6. SCTP(Stream Control Transmission Protocol): 流控制传输协议，为了进行视频和音频的传输

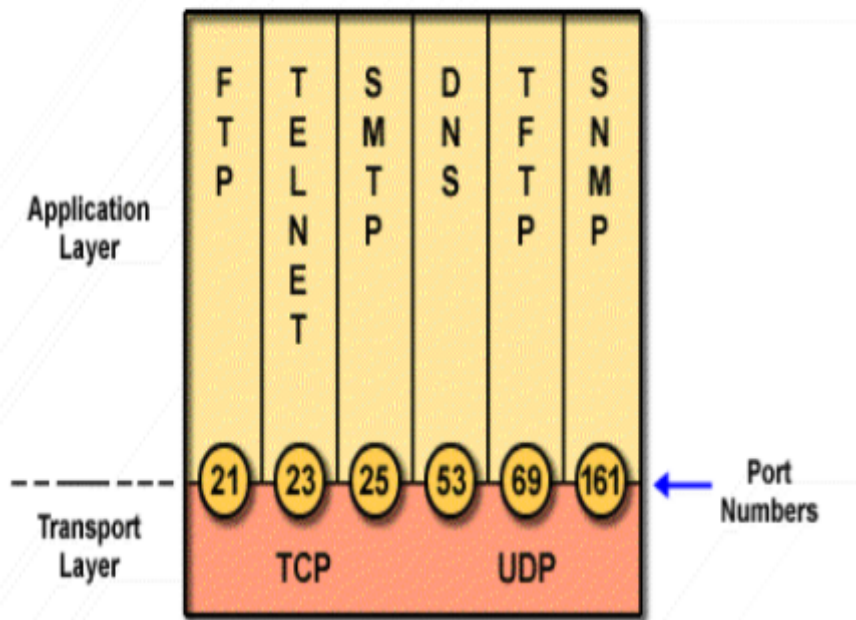
1.1. 服务模型

一个主机 不同进程 复用IP

1. TCP和UDP都使用端口来跟踪(track)同时穿越网络的不同会话
2. 应用软件开发人员已同意使用RFC1700中定义的知名端口号

操作系统用port识别不同上网进程

Port Numbers



1.1.1. 端口分配规范

1. 低于255的端口号(0-255)保留给TCP和UDP公共应用程序使用。(端口号0-255是public的，不可以随意分给其他的进程，如果分发则不符合规范)
2. 0-1023是熟知端口，有分发的规范，不应当被随意使用
3. 1024-49151的端口号进行登记使用，有的是应用程序已经的使用端口号，避免冲突
4. 基于端口号的不同，进行不同的包的分发

49152-65535 客户端进程使用 短暂端口号

1.2. 套接字(Socket, 第四层的单位)

通讯.

1. 套接字表示为 (IP地址, 端口)
2. 每个连接都表示为 (socket_{source}, socket_{destination})，这是一个点对点全双工通道
3. 通讯被认为是以一个socket和另一个socket之间的连接。(Socket API是一套规范，根据上下文有不同的含义)
4. TCP不支持多播和广播

2. TCP (Transmission Control Protocol)

2.1. TCP必须解决的问题

- 1. 可靠传输
- 2. 流传输
 - 1. 滑动窗口(窗口进行通信，一次数据传输是有上限发的，缓存问题，拥塞问题)
 - 2. 避免拥塞
- 3. 连接控制
 - 1. 建立连接:三次握手
 - 2. 断开连接:四次握手

2.2. TCP数据段的格式



2.2.1. 首部情况

一行共计4字节，段首在前，固定首部长度为20字节。

2.2.2. 源端口和目的端口

源端口和目的端口字段:各占 2 字节

- 1. 端口是运输层与应用层的服务接口
- 2. 运输层的复用和分用功能都要通过端口才能实现

2.2.3. 序号字段

序号字段：占 4 字节(4×2^{32})的地址空间)

1. TCP 传送的数据流中的每一个字节都编上一个序号
2. 序号字段的值指本报文段所发送的数据的第一个字节的序号
3. 通过序号字段做可靠传输的保证，指示的是一个TCP传输的bit编码，而不是地址。
4. 我们从小向大进行使用，如果使用到最大之后，我们会从小再次重新开始分配。

2.2.4. 确认号字段

确认号字段：占 4 字节，是期望收到对方的下一个报文段的数据的第一个字节的序号

1. 确认对方的数据号(发送同时对上一次传输进行确认)
2. 体现出了全双工通信的优点，比如上回收到最后序号是700，那么确认号就是701

2.2.5. 数据偏移

数据偏移（即首部长度的）：占 4 位 *1~15*

1. 指出 TCP 报文段的数据起始处距 TCP 报文段的起始处的长度(Data部分从什么地方开始算)
2. 单位是 32 位字（以4 字节为计算单位）*(行)*
3. 不满足的话使用填充位保证为4字节的整数倍(保证对齐问题)

2.2.6. 保留字段

保留字段：占 6 位，保留为今后使用，目前置 0，也就是说截止到现在也没有使用这部分的字段。

2.2.7. URG *urgent*

紧急 URG = 1 时，表明紧急指针字段有效。

1. 告诉系统此报文段中有紧急数据，应尽快传送(相当于高优先级的数据)
2. 优先放紧急数据，0 的时候则为不紧急(Ctrl + Z)

2.2.8. ACK

ACK = 1 时确认号字段有效；ACK = 0 时确认号字段无效

2.2.9. PSH(PuSH)

推送

1. 接收 TCP 收到 $PSH = 1$ 的报文段，就尽快地交付接收应用进程，而不再等到整个缓存都填满了后再向上交付，此时将缓存所有部分都传输，而并不是只将这个报文段的信息进行传输。
2. TCP在正常条件下并不是立马传输的，首先要缓存满了才发送，其次还有就是要保证网络可信的时候才发送

2.2.10. RST 复位

1. $ReSeT = 1$ 时，表明TCP连接中出现严重差错（如由于主机崩溃或其他原因），必须释放连接，然后再重新建立运输连接
2. 就是重新来过，如果请求方发送的请求，如果应答方不想连接则将 $ReSet$ 置为1

2.2.11. SYN 同步 连接确认 $SYN = 0$

同步 $SYN = 1$:表示这是一个连接请求或连接接受报文(初始的时候才出现)

2.2.12. FIN(FINis) 终止

用来释放一个连接。 $FIN = 1$ 表明此报文段的发送端的数据已发送完毕，并要求释放运输连接。(发送方没有传输数据了)

2.2.13. 窗口

1. 占 2 字节，用来让对方设置发送窗口的依据，单位为字节。
2. 表示可以进行传输的窗口大小是多少。允许对方发送的数据量

2.2.14. 检验和

检验和:占有2字节。

1. 检验和字段检验的范围包括首部和数据这两部分
2. 2字节，IP报文中的地址等伪首部进行校验

2.2.15. 紧急指针字段

紧急指针字段:占有2字节

1. 指出在本报文段中紧急数据共有多少个字节（紧急数据放在本报文段数据的最前面）

2.2.16. 选项

1. TCP 最初只有一种选项，即最大报文段长度 MSS(Maximum Segment Size)

根据终端
冲突情况
↓
最终以窗口
为准

首部的内容

初始协商时
根据网络情况

2. MSS 告诉对方缓存所能接收的报文段的数据字段的最大长度是 MSS 个字节
3. 数据字段加上 TCP 首部才等于整个的 TCP 报文段

2.2.17. 填充字段

填充字段：这是为了使整个首部长度是 4 字节的整数倍。

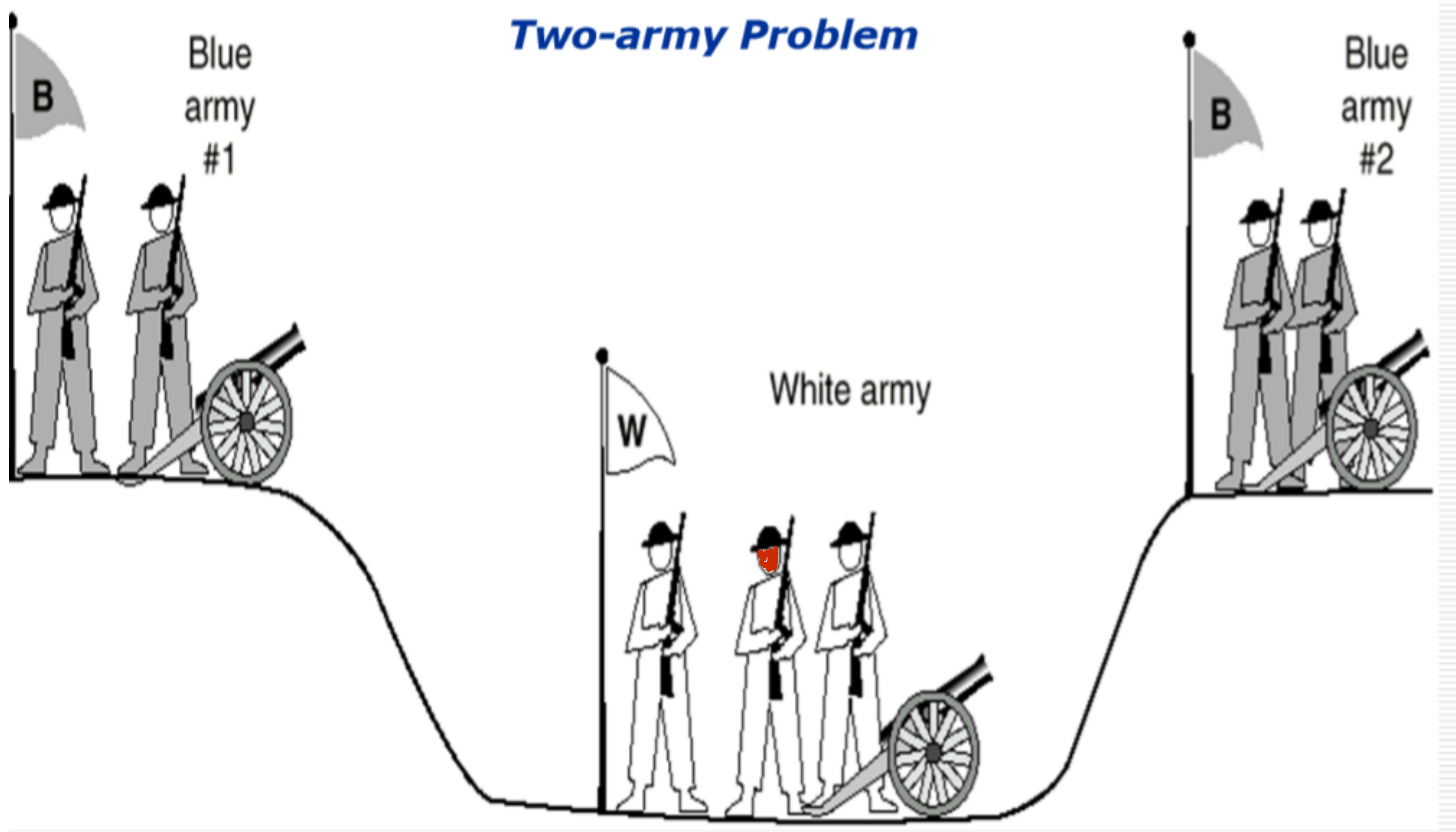
2.3. TCP 协议

Transport Protocol Data Unit.

1. 主机使用网段（TPDU）交换数据
2. 每个段都有：
 1. 标头为20个字节（可选部分除外）
 2. 0或更多数据字节(请求连接的时候)
3. 段的大小必须与IP数据包匹配，并且还必须满足底层的需求
 1. 例如，以太网的MTU（最大传输单位）为1500字节
 2. 是面向字节的传输。
4. 每个字节都有一个32位序号
 1. 通讯中商定初始序号，确认到每一位
 2. 面向字节:TCP传输的数据块和上层数据给的数据块的大小可以不对应(通过商量解决)
 3. 根据网络条件，对每一个字节进行确认，

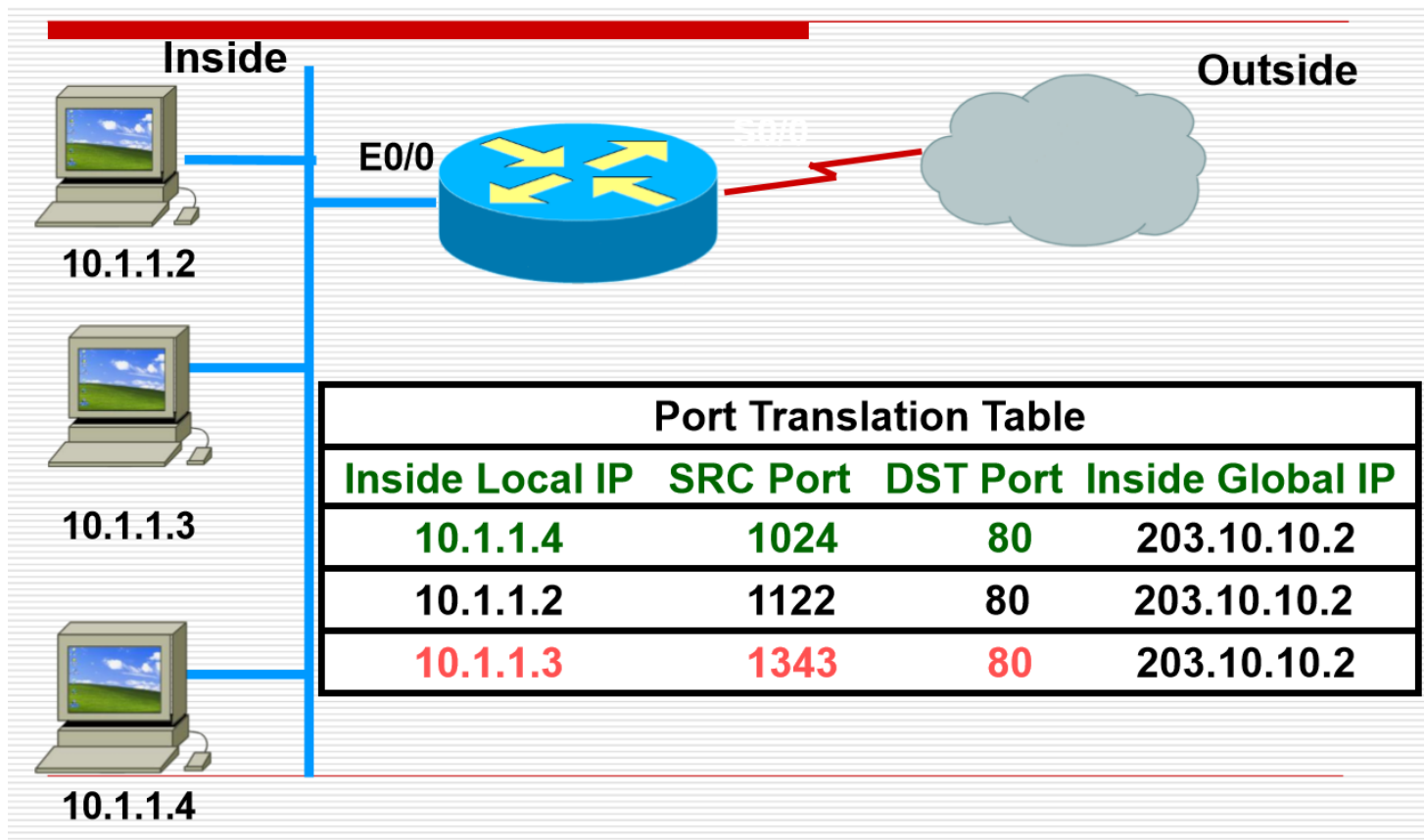
2.3.1. Reliable Connection 可靠连接

握手机制

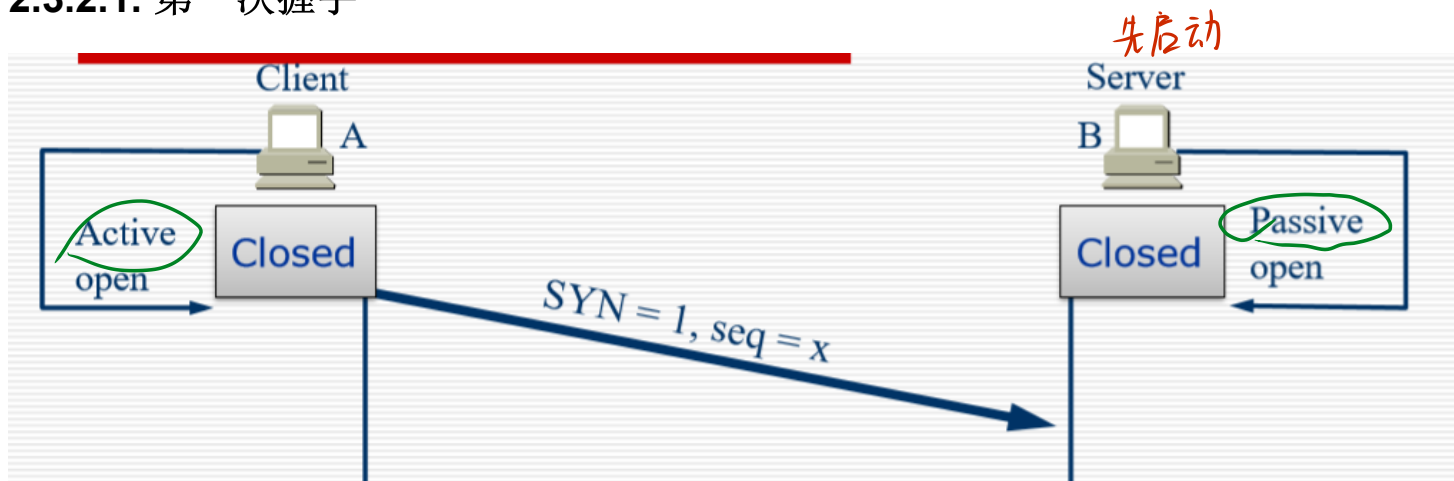


1. 红蓝两军问题
2. 两军之间传输信息，由侦查员进行传递
3. 结论:无论通信多少次，都不能确定一个完全可信的时间。

2.3.2. TCP: Establish Connection TCP:建立可靠连接

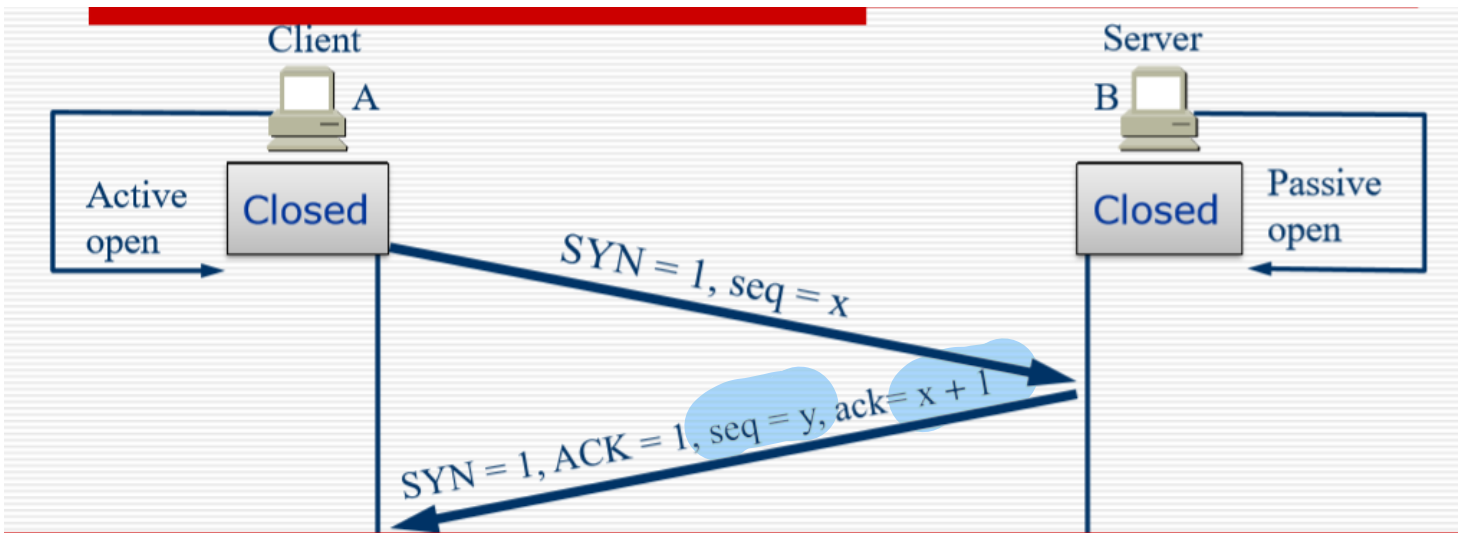


2.3.2.1. 第一次握手



1. 服务器：执行LISTEN和ACCEPT原语，并进行被动监视
2. 客户端：执行CONNECT原语，生成SYN = 1和ACK = 0的TCP段，代表连接请求

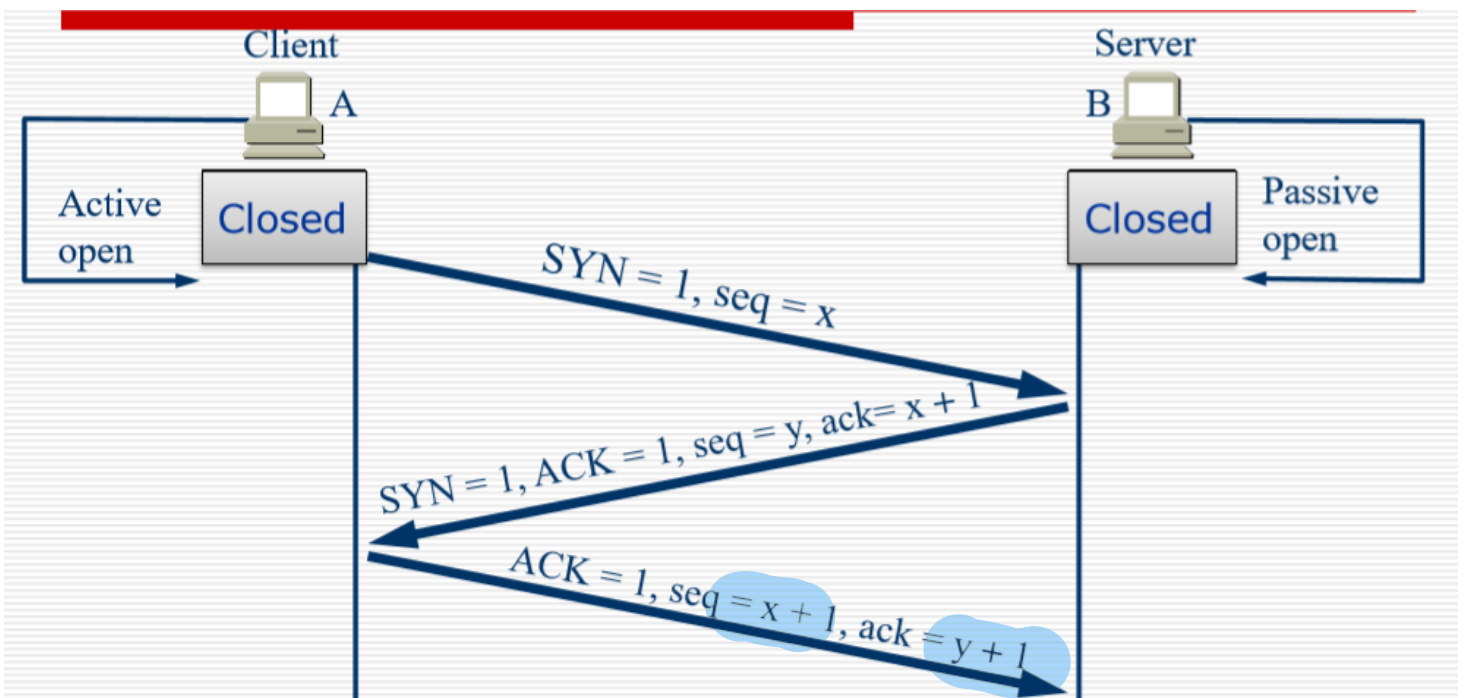
2.3.2.2. 第二次握手



1. 服务器检查是否存在监视端口的服务进程

1. 如果没有任何进程，请使用 $RST = 1$ 回答一个TCP段
2. 如果存在进程，则决定拒绝或接受请求
3. 如果接受连接请求，则发送 $SYN = 1$ 和 $ACK = 1$ 的网段

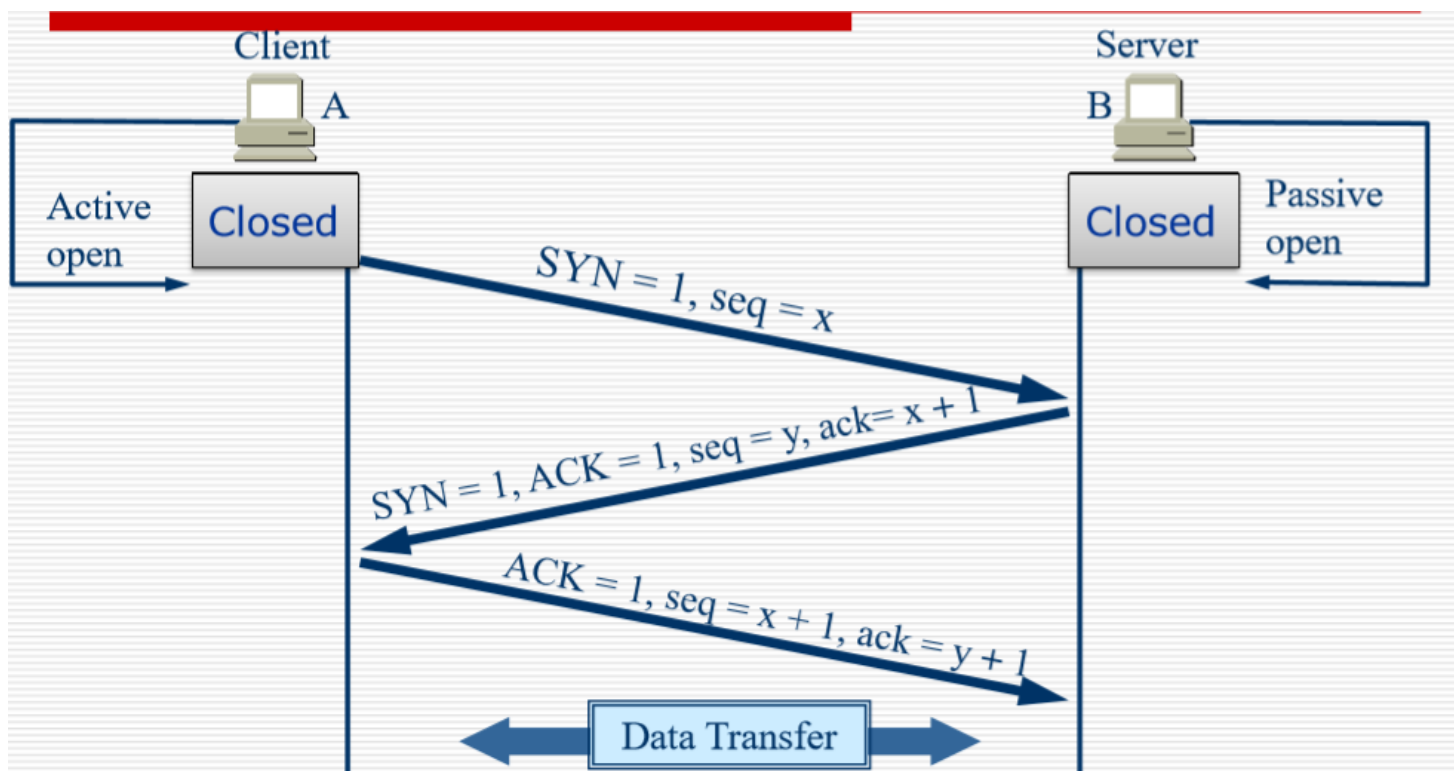
2.3.2.3. 第三次握手



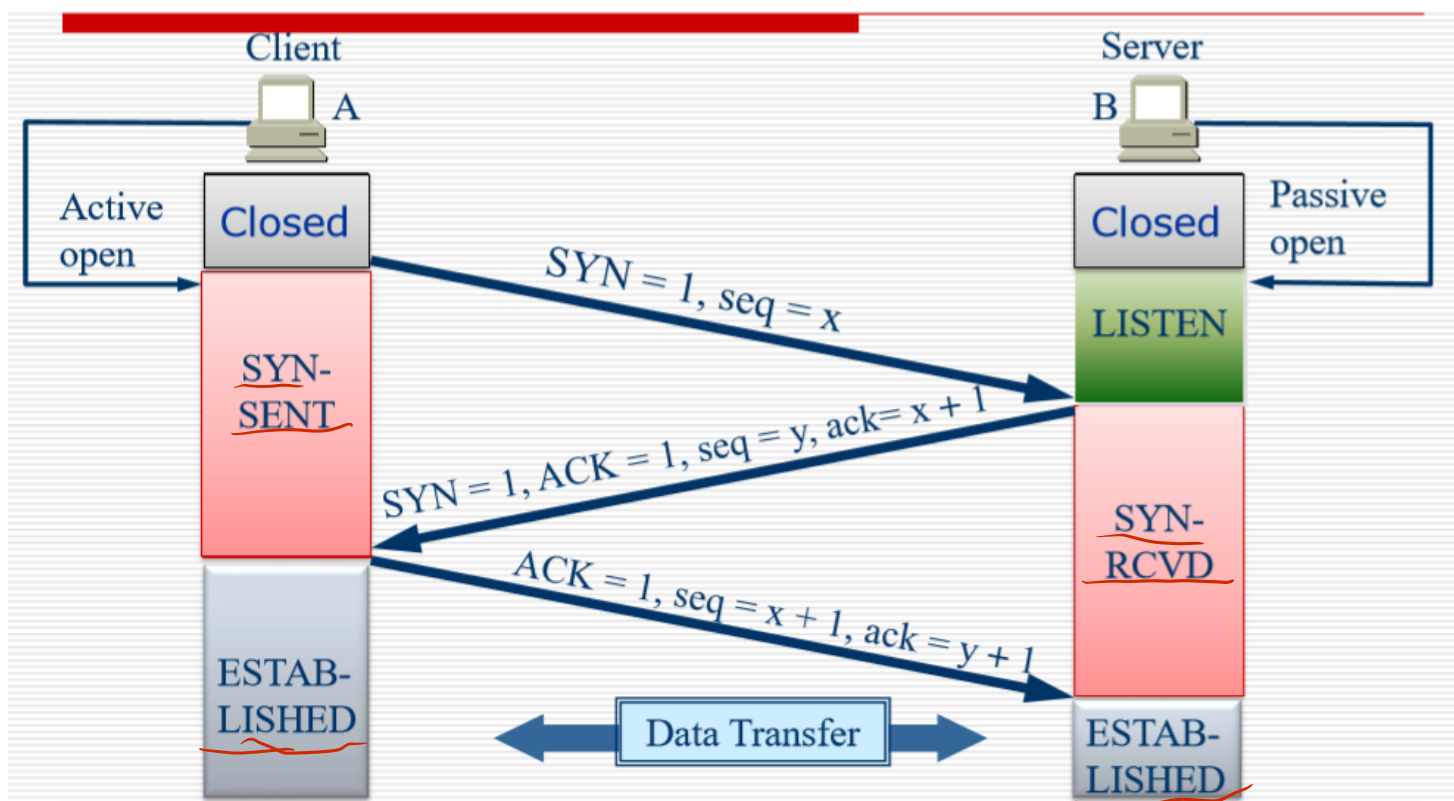
1. 客户端发送一个 $SYN = 0$ 和 $ACK = 1$ 的段以确认连接

2. 为了避免出现延时之类(如果只有两次会浪费服务器资源)

2.3.2.4. 通知上层应用，准备数据传输

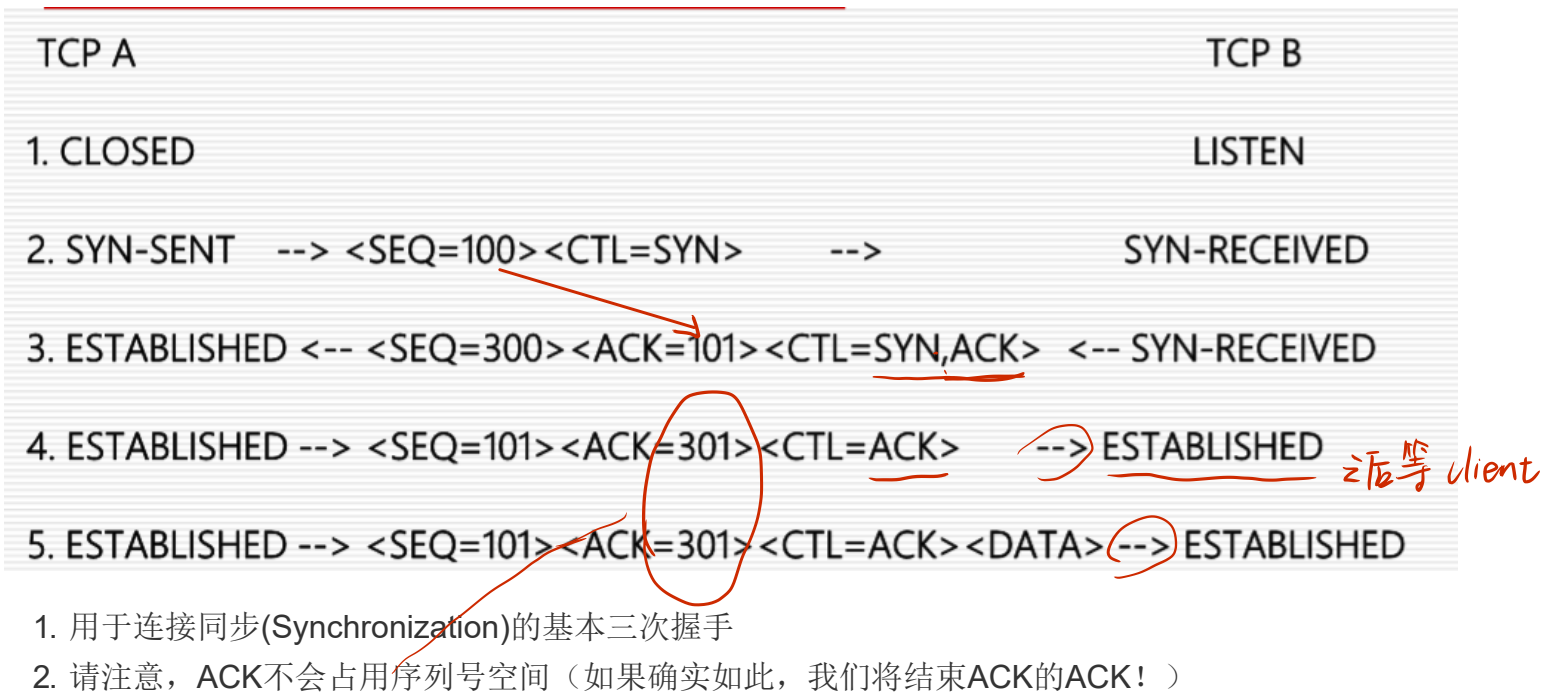


服务器收到确认后，会通知上层应用程序

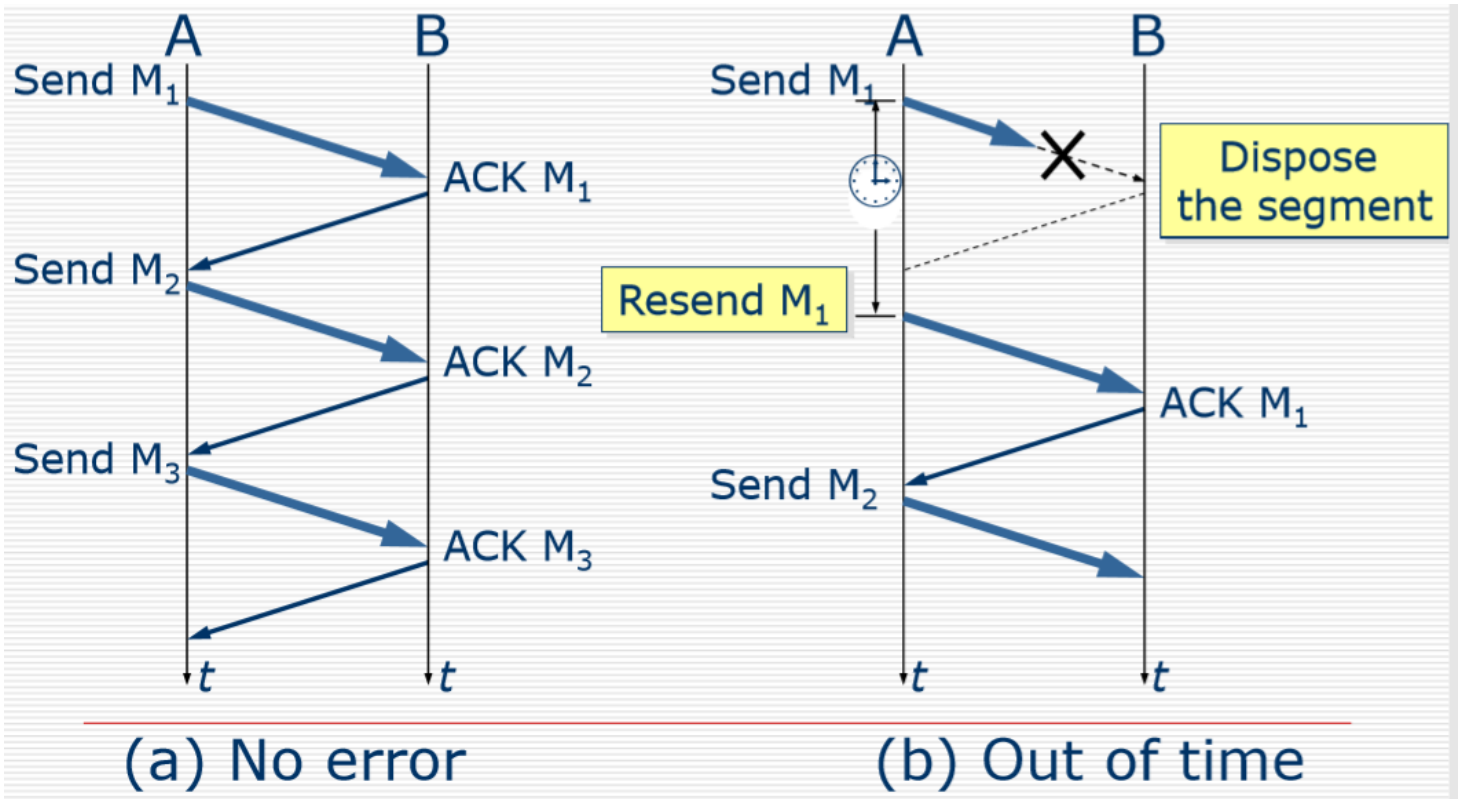


1. 默认三次握手就认为可靠了，之后就进行数据传输
2. 有时候我们会选择，第三次握手的时候同时携带数据。

2.3.3. 建立连接实例



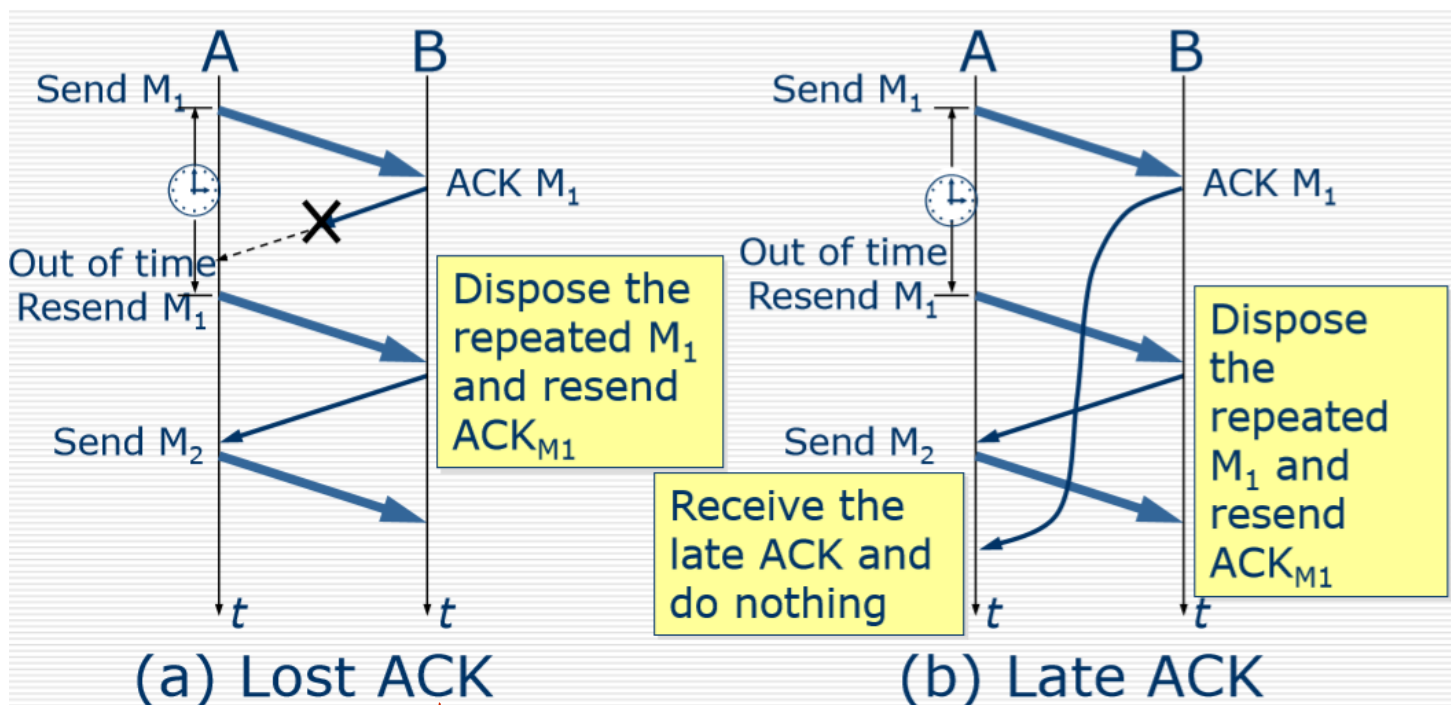
2.4. 可靠传输 (停止等待协议) stop-and-wait protocol



1. 发送段后，暂时保留备份 (可能重发)
 1. 在发送后没有收到确认的时候，要保存备份来重传
 2. 收到确认的时候，抛弃备份
 3. 超时计时器：如果对方的应答超过一定时间后则直接进行重发(时间要比正常往返时间稍微长一点)

2. 每个网段和ACK必须具有ID
3. 重新发送时间必须大于平均传输时间 * 2
4. 停止等待协议是一个简单的协议，但是效率很低
5. 实施控制，来进行错误处理

2.5. 数据传输 - 丢失确认 和 确认延迟



1. 发过去没有应答或者丢失:进行重传
2. 应答超时，有收到请求立即重传

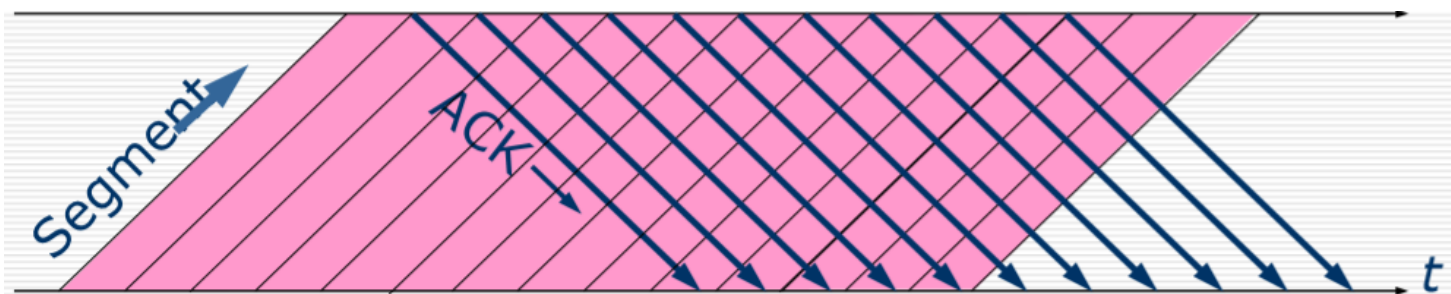
发迟序. 晚到的应答直接丢弃(不做处理)

2.6. 可靠通信

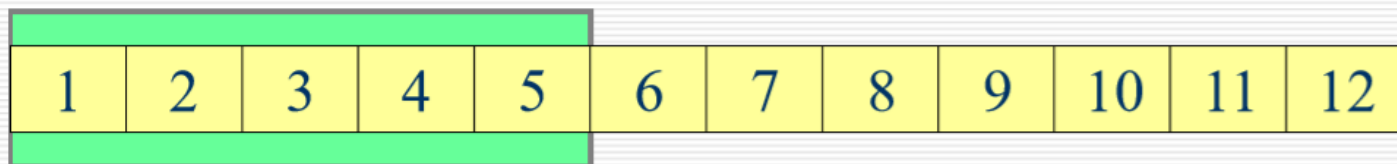
1. ARQ (Automatic Repeat reQuest) 自动重传请求: 这表示“重新发送请求”为自动发送并且接收方无需请求发送方重新发送错误段

提高传输效率

2.6.1. Contiguous ARQ(Automatic Repeat-reQuest) Protocol 连续ARQ协议

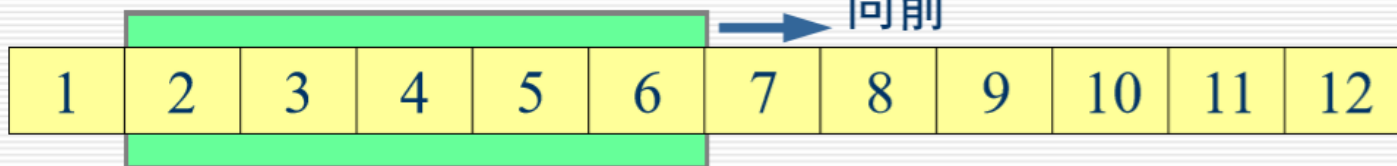


发送窗口



(a) 发送方维持发送窗口（发送窗口是 5）

发送窗口

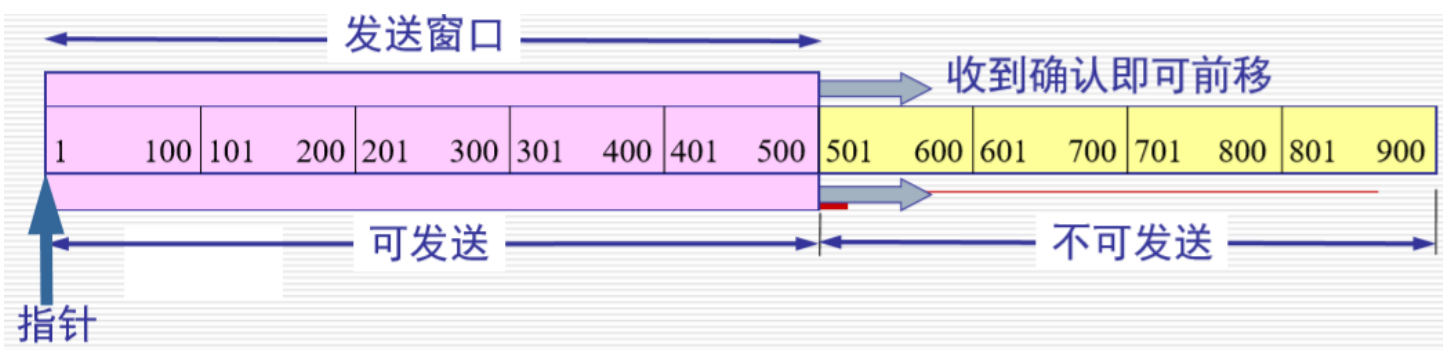


(b) 收到一个确认后发送窗口向前滑动

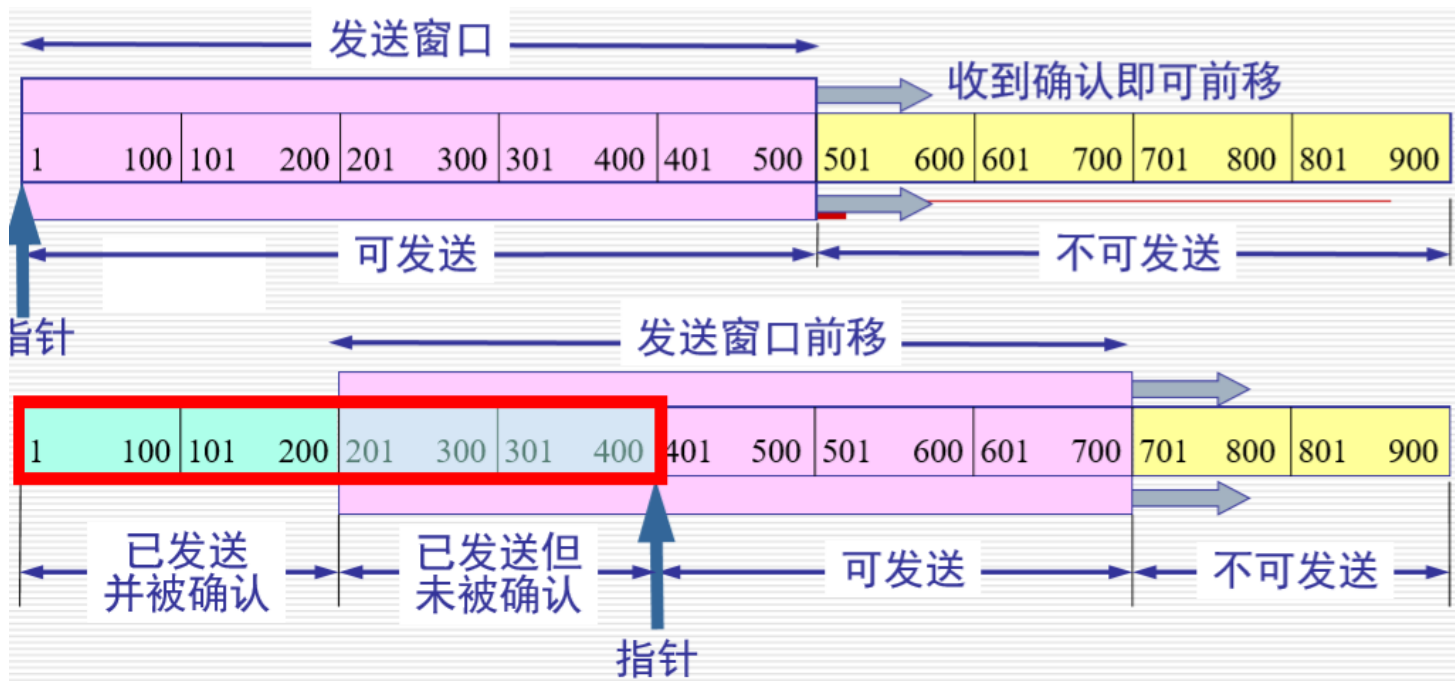
1. 多个数据同时发送过去(一次发送多个)
2. 窗口大小是双方协商的，通过TCP报文中的窗口字段表示。

可作调整

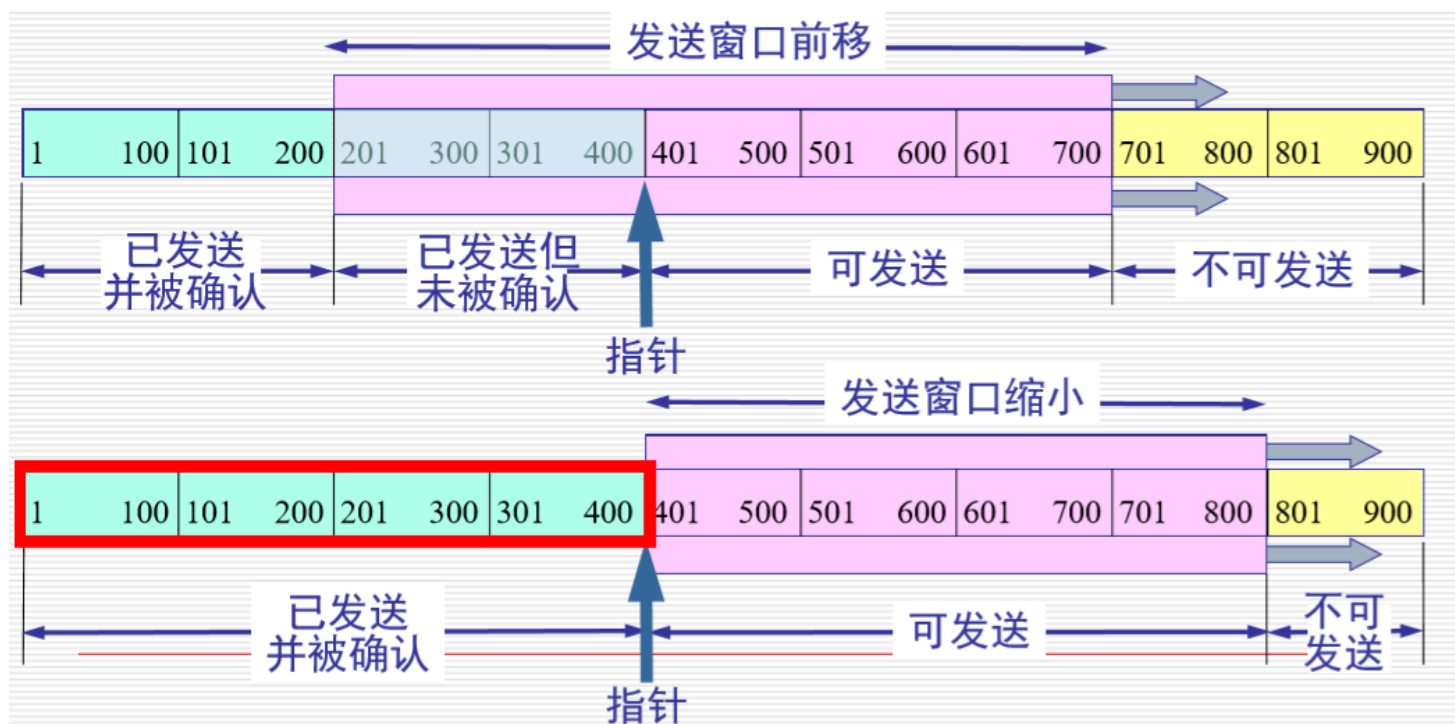
2.6.2. ARQ具体实例



1. 发送端要发送 900 字节长的数据，划分为 9 个 100 字节长的报文段，而发送窗口确定为 500 字节。
2. 发送端只要收到了对方的确认，发送窗口就可前移。
3. 发送 TCP 要维护一个指针。每发送一个报文段，指针就向前移动一个报文段的距离。



4. 发送端已发送了 400 字节的数据，但只收到对前 200 字节数据的确认，同时窗口大小不变。
5. 现在发送端还可发送 300 字节。
6. 发送端收到了对方对前 400 字节数据的确认，但对方通知发送端必须把窗口减小到 400 字节。
7. 现在发送端最多还可发送 400 字节的数据。



8. 利用可变窗口大小进行流量控制双方确定的窗口值是 400

又一个例子

为什么?

为什么?

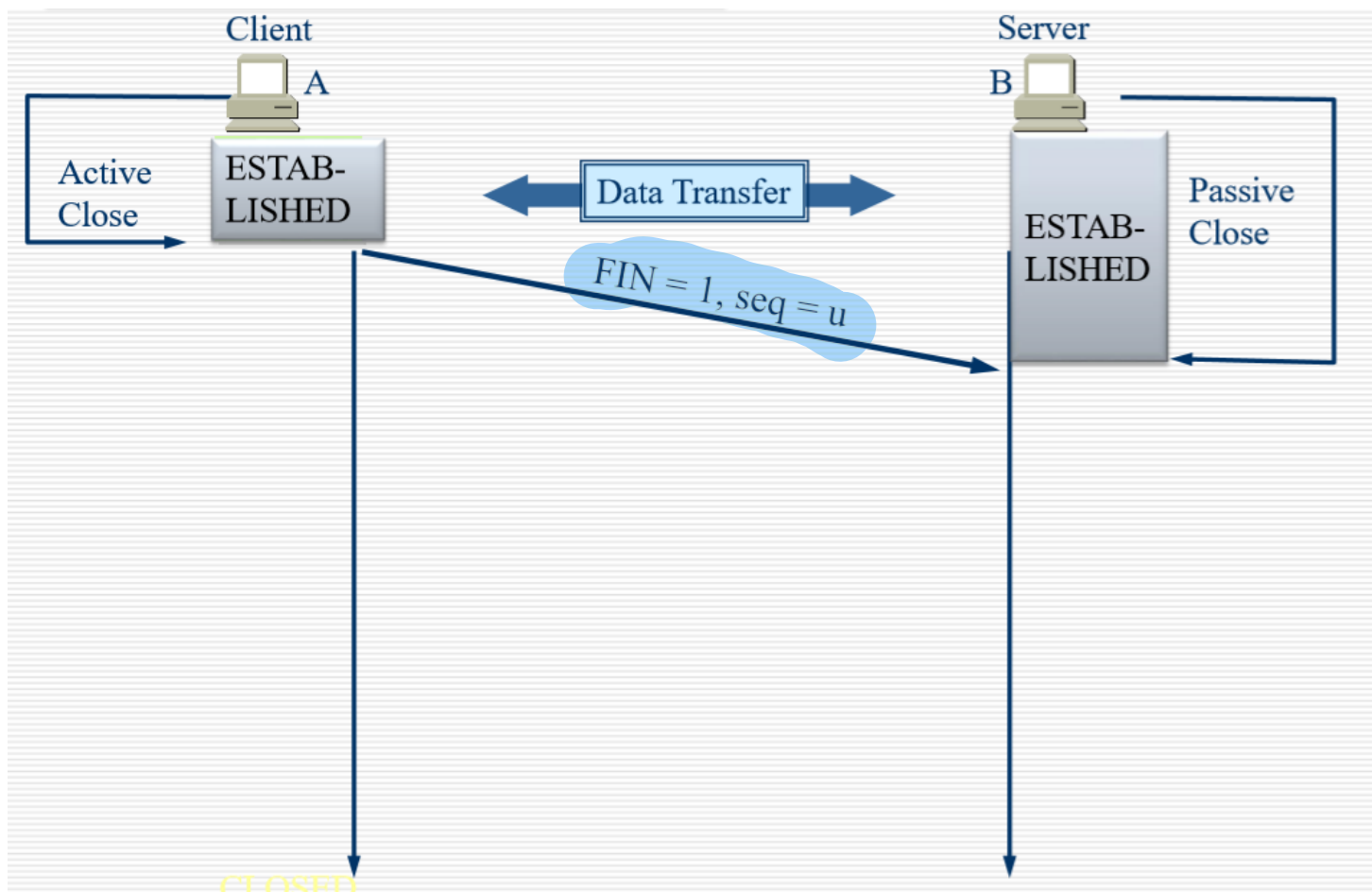


9. WIN:窗口的大小:双方动态协商, 收到确认调整窗口

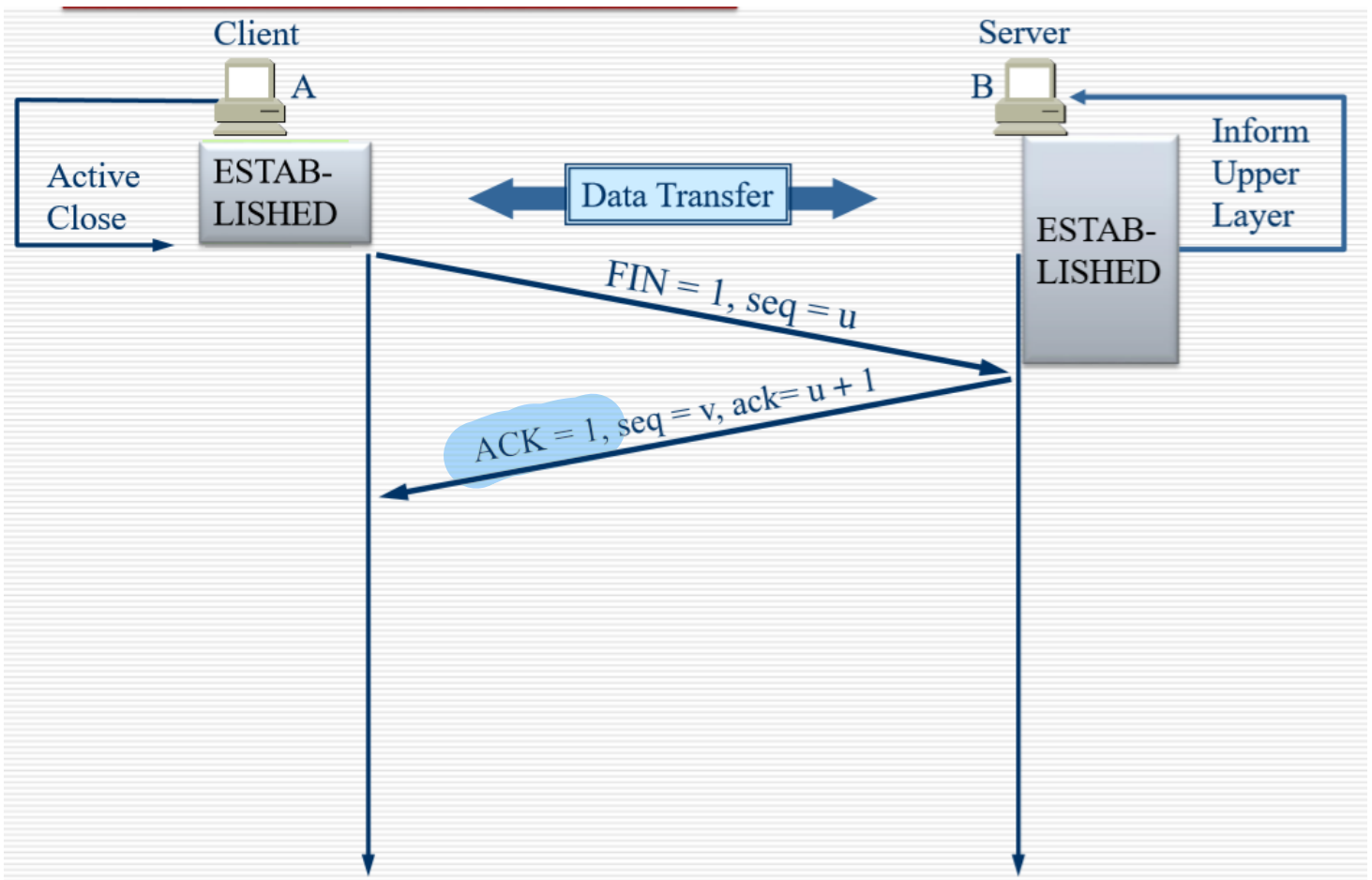
10. ACK:是指可以继续发送的数据的位置。

11. 为什么201在401后面发送? 超时重传(要超过两倍的平均传输时间后才进行重传)

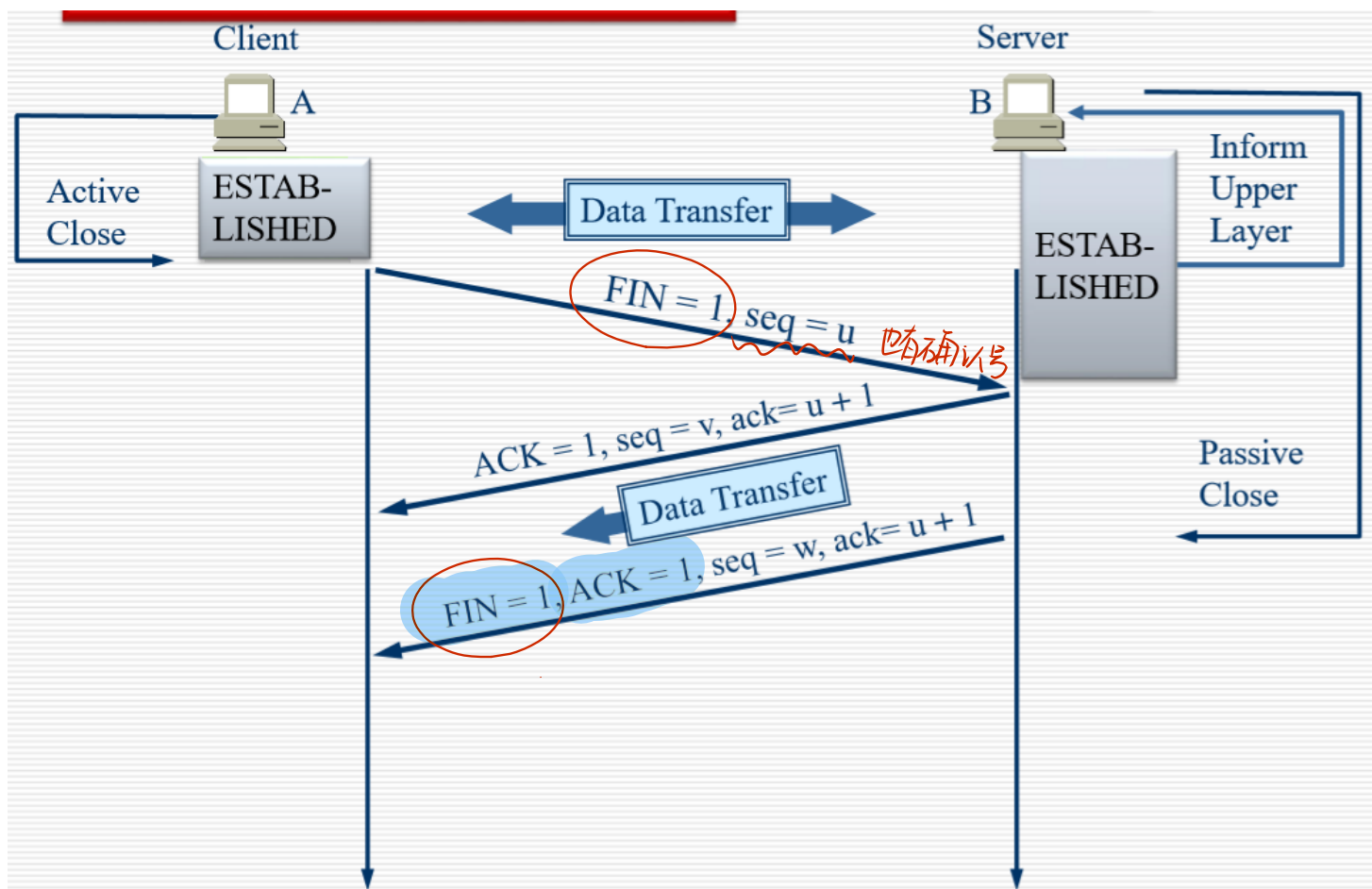
2.7. TCP:释放链接



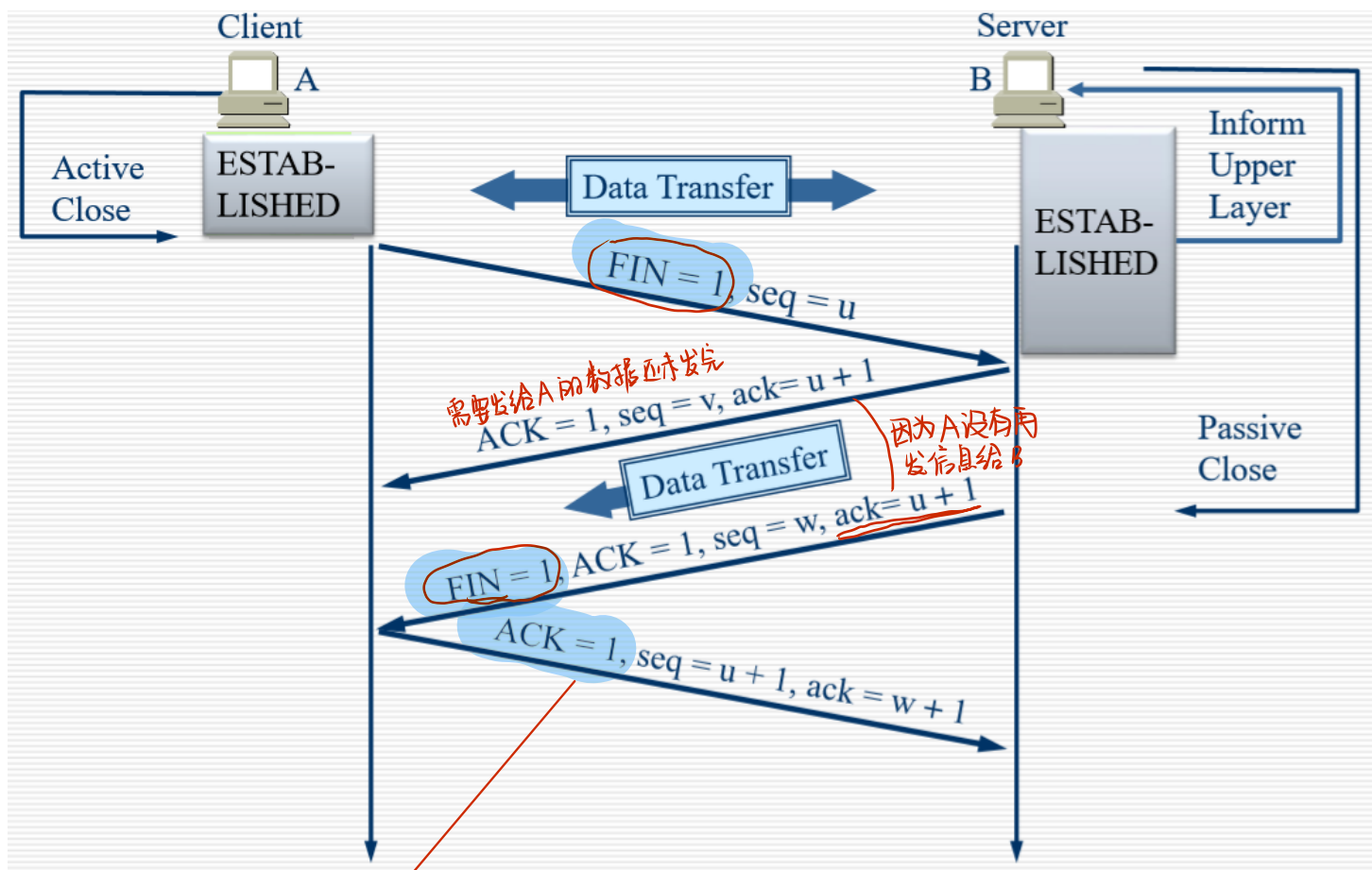
1. 发起断开连接请求



2. $Ack = 1$: 允许断开，但是此时并不是断开连接，而是说不在发送新的数据，此时我们需要完成之前未处理完成的数据的处理。(这里只是说我已经收到了你请求停止传输的请求)

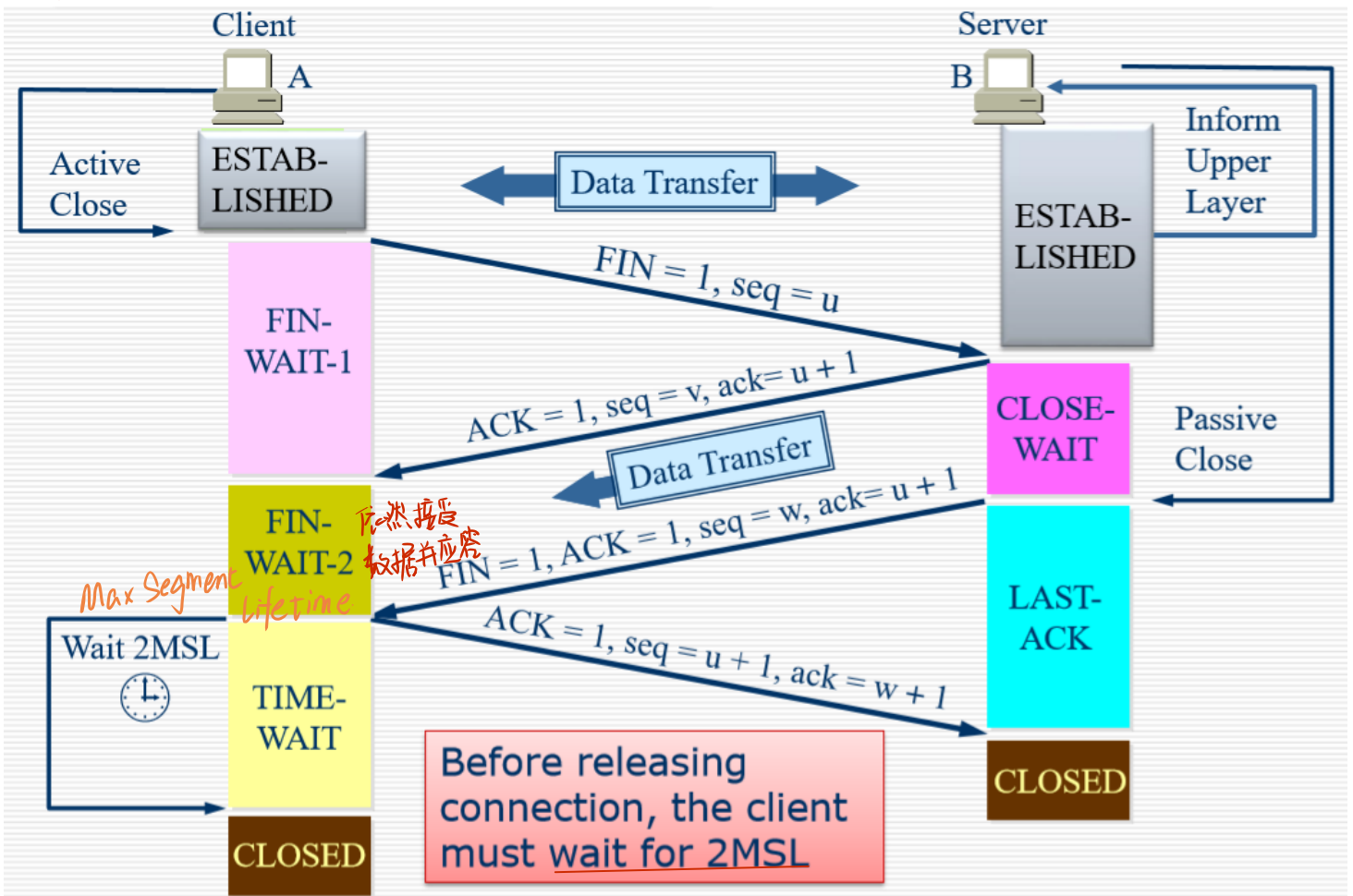


3. **FIN = 1**:数据处理完成，注意需要的变化(此时表示所有的需要处理的数据已经处理完了，此时表示正式确认断开)



4. 确认收到B的断开信息

状态



5. 上图是释放连接的汇总

1. 等待最大的网路往返时间(保证能处理到B最后发送的报文)

比如B没有收到A的应答 会超时重发

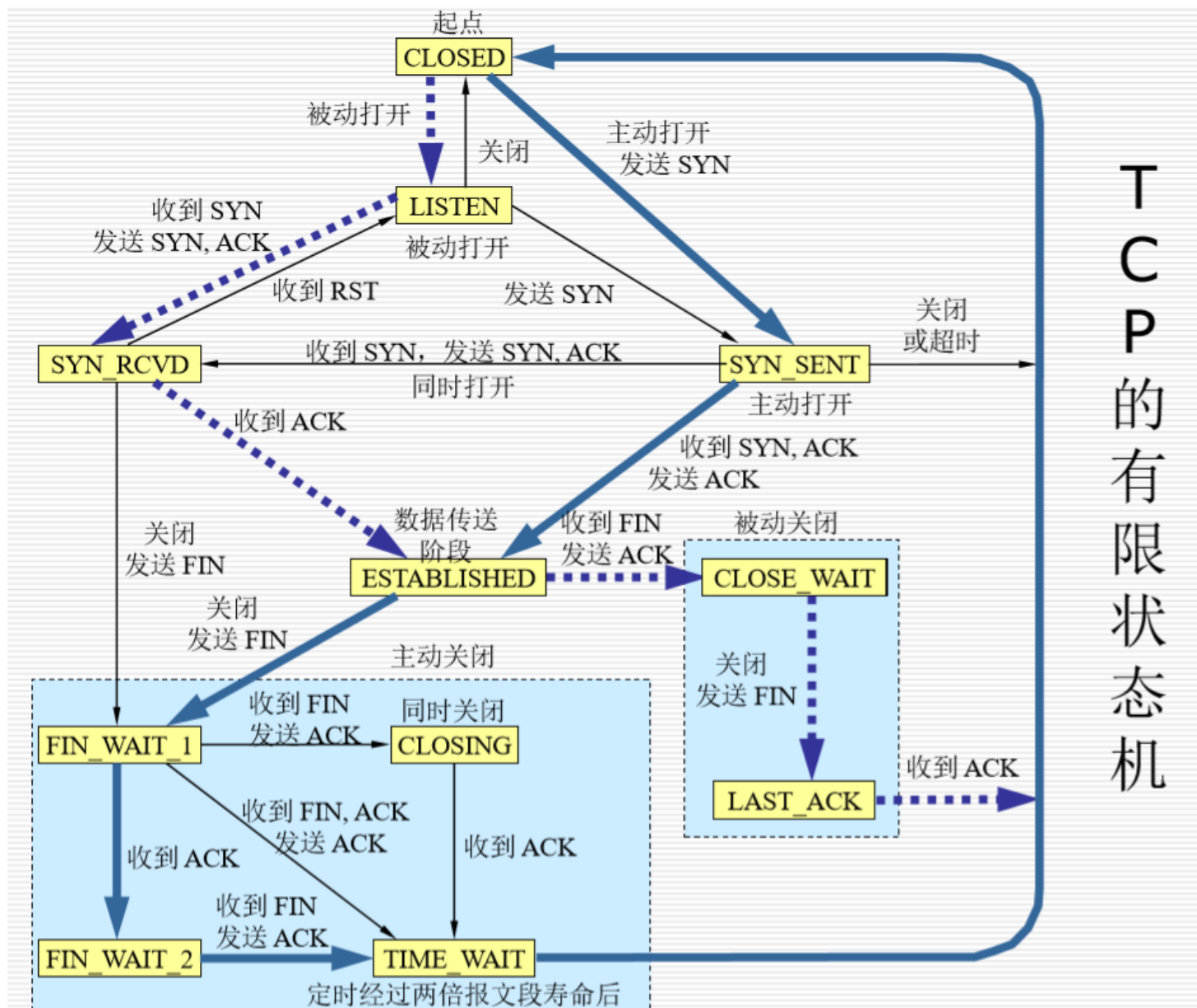
2.8. 为什么必须等待2MSL?

1. 为了确保A发送的最后一个ACK可以到达B
2. 防止出现任何无效的连接请求段: 等待2MSL之后, 我们可以确保连接上的所有段均已消失

2.9. TCP中的计时器

1. 重传计时器: 多长时间进行重传
2. 坚持计时器: 避免死锁(WIN = 0的时候修改WIN但是没有成功发送过去): 收到WIN = 0的时候 开始进行计时, 到时间主动询问
3. 保持计时器:
 1. 发送数据段后, 刷新
 2. 如果到达一定的时间, 则再次询问是不是还要保持连接。
4. 时间等待计时器

2.10. TCP的有限状态机



1. 粗线: 正常的服务器端 (粗蓝色箭头)
2. 虚线: 正常客户端 (虚蓝色箭头)
3. 细线: 异常状态的问题 (细蓝色箭头)

3. 用户数据报协议(UDP, User Datagram Protocol)

3.1. 用户数据报协议UDP (User Datagram Protocol)

1. 为什么我们需要UDP?

1. 没有建立连接（避免延时）
2. 简单：发送方，接收方无连接状态
3. 小段标题 **头部**
4. 没有拥塞控制：UDP可以按照期望的速度传输

2. 无连接：没有复杂控制，头部简单

1. UDP发送方，接收方之间没有握手(HandShake，包含进程等信息的)
2. 每个UDP段都独立处理 也无“确认”

3. 常用于流媒体(Stream)多媒体(multimedia)应用

1. 容忍损失:无非就是降低帧率
2. 这类应用是速率敏感的应用，而不一定是质量敏感的应用。

4. UDP用于:

1. RIP:定期发送路由信息(periodically) 见上一章 **一种动态路由协议**
2. DNS:避免延迟建立TCP连接(DNS需要快速找到) **域名?**
3. SNMP:拥塞时(congestion), SNMP必须仍然可运行。在没有拥塞和可靠性控制机制的情况下，UDP在这种情况下性能要优于TCP。(主播和多播，大量信息传输) **简单网络管理协议**
4. 其他协议包括TFTP, DHCP **简单文件传输** **动态主机配置协议**

5. 必要时增加应用层的可靠性措施

6. 流媒体就算有数据丢失也问题不大(对屏幕进行模糊化处理就行),但是发送速率是很重要的! (就算丢包了，也可以模糊处理)

3.2. UDP数据帧格式

UDP Segment Format

# Bits	16	16	16	16	
	Source Port	Destination Port	Length	Check-sum	Data ...

- No sequence or acknowledgement fields

↓
用于握手

1. UDP的数据段很简单
2. UDP只有8个字节的首部 *有效长度多(占比1-)*
3. 源端口、目的端口、长度、校验(data)、Data
4. 校验也要对data一并校验，如果出现错误，直接丢弃。
5. 应用层进行数据切片，决定如何进行发送，UDP直接发送

3.3. TCP和UDP的不同

3.3.1. TCP

1. 不是立即交给上层校验，而是需要先和对方沟通 *(窗口条件. 面对字节流)*
2. 缓存满了才统一交付。

3.3.2. UDP

1. 直接转发报文，保留报文边界
2. IP进行划分
3. 应用程序会发送比较合适的UDP报文大小进行发送

3.3.3. 共同点

1. 校验是相同的。

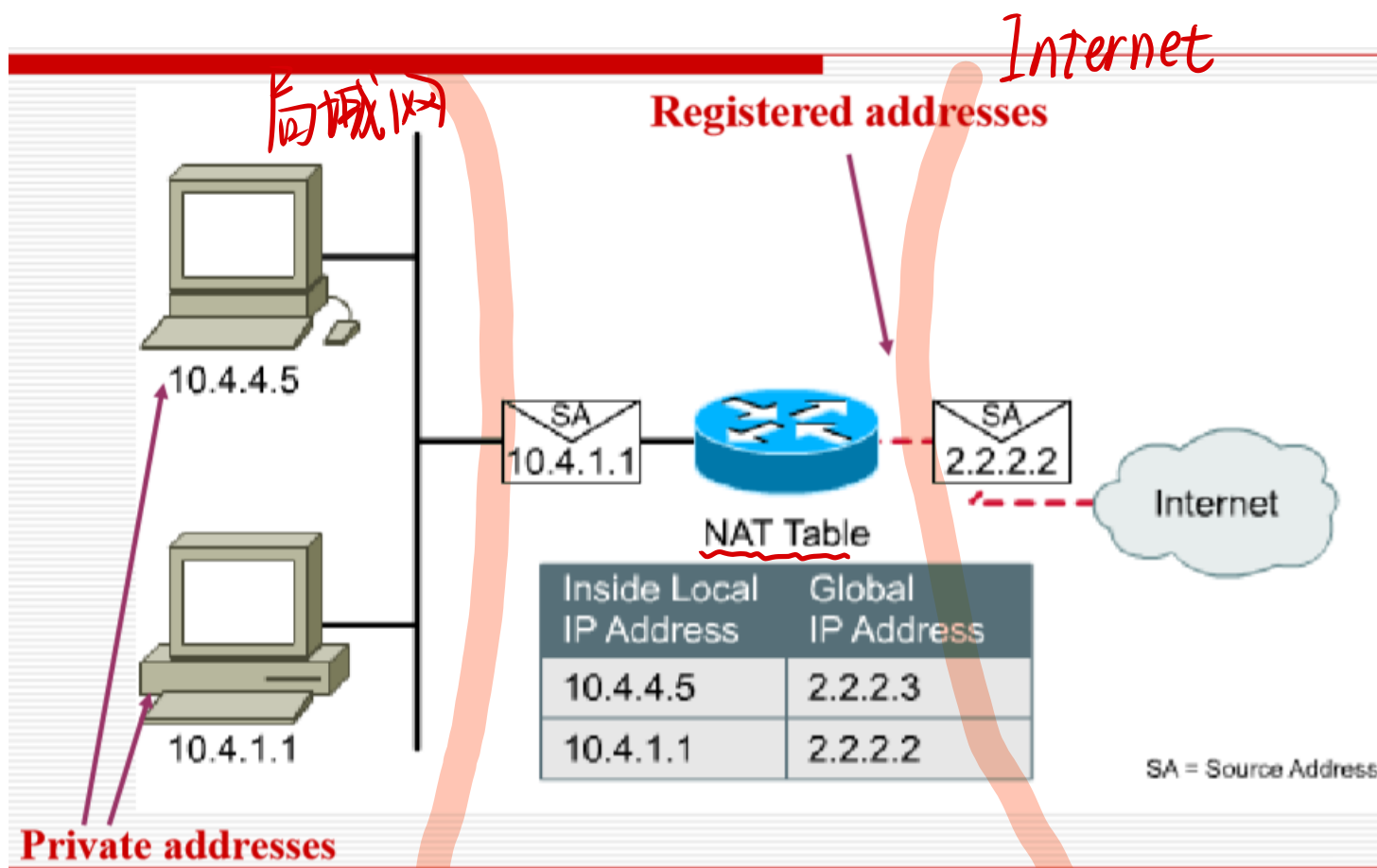
4. 应用：NAT和PAT

网络地址转换.

4.1. 什么是NAT? Network Address Translation

1. NAT，是在IP数据包头中将一个地址交换为另一个地址的过程
 1. 网络地址转换
 2. 是网络地址即将用完的解决方案
2. 实际上，NAT用于允许私下寻址的主机访问Internet. *private IP地址 访问 Internet*
3. IP地址耗尽的解决方案之一
 1. 保留注册（合法）地址
 2. 连接到Internet时增加灵活性
4. RFC 1631 - Network Address Translator (NAT)

4.2. NAT技术是一个简单的概念



1. NAT需要一个路由器来实现
2. 左侧是一个局域网
3. 在NAT 路由器将局部地址 转换成 网络上的地址(双向转换, 有一个NAT表)

private global

4.3. NAT的类型

1. 静态NAT: 固定的内部地址(internal address)到注册地址(registered address)的映射(一开始就写死)
2. 动态NAT: 映射以先到先得的方式动态进行(不是写死, 配一个地址池, 通过更新)
3. PAT (过载, Port address translation): 端口地址转换用于允许许多内部用户共享一个“内部全局”地址(基于Socket映射, 而不是IP地址, 多个内网主机映射到一个公网地址)

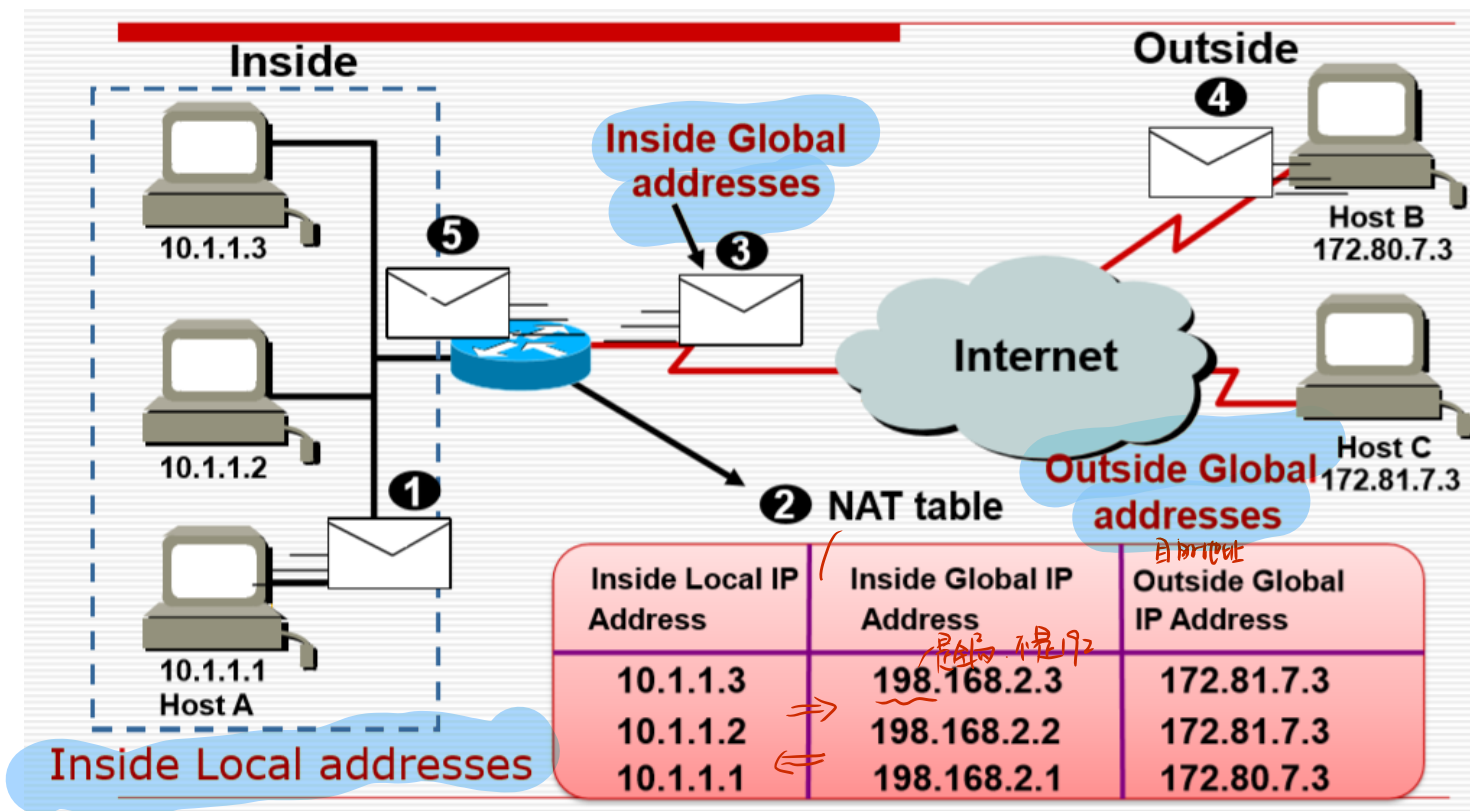
Overload

端口 port

全局
1个IP - 多个端口

4.4. NAT地址类型

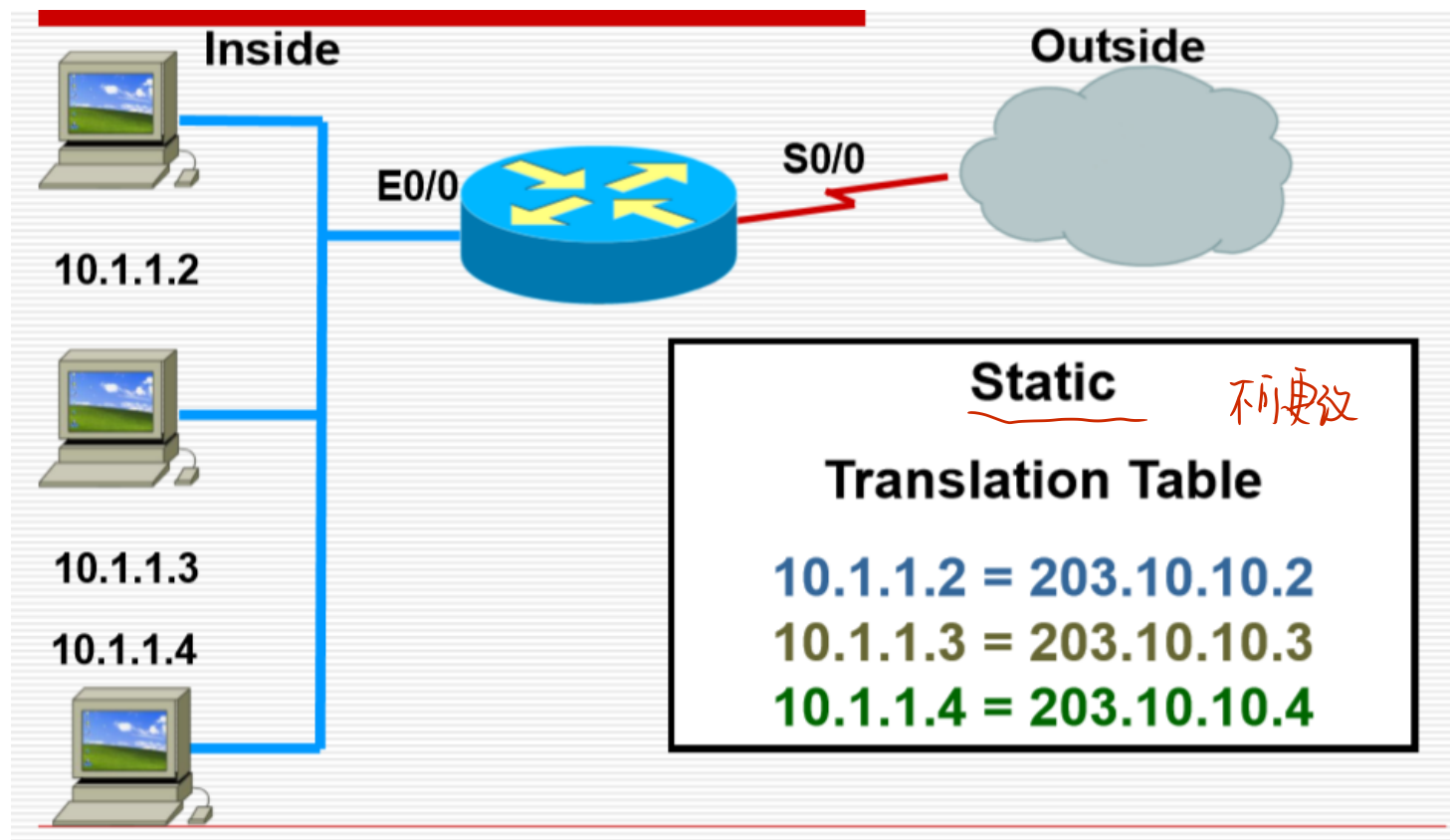
1. Inside Local address (内部本地地址): 内网IP地址 即 private IP
2. Inside Global address (内部全局地址): 注册IP地址, 对外部展示的内部地址 NAT转化后的
3. Outside Global address (外部全局地址): 由主机所有者分配的IP地址。通常是注册地址。(对内网而言的外部, 是目的地址)



4. 三个地址在上面可以看一下

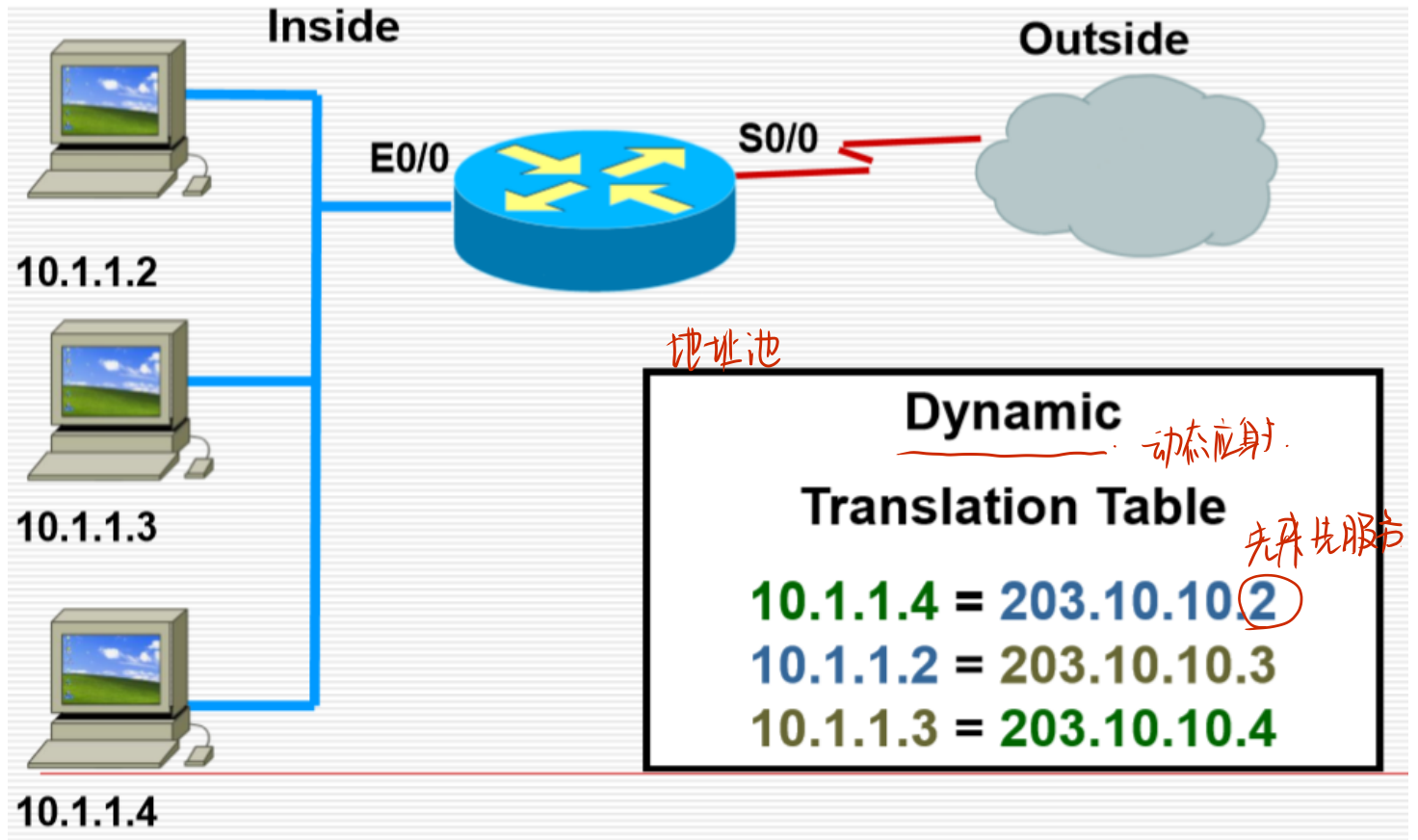
5. 内部主机发送报文给网关，网关根据NAT Table进行翻译，转换成内部全局地址，然后进行转发

4.5. 静态NAT的例子



1. 这是静态的表

4.6. 动态NAT的例子



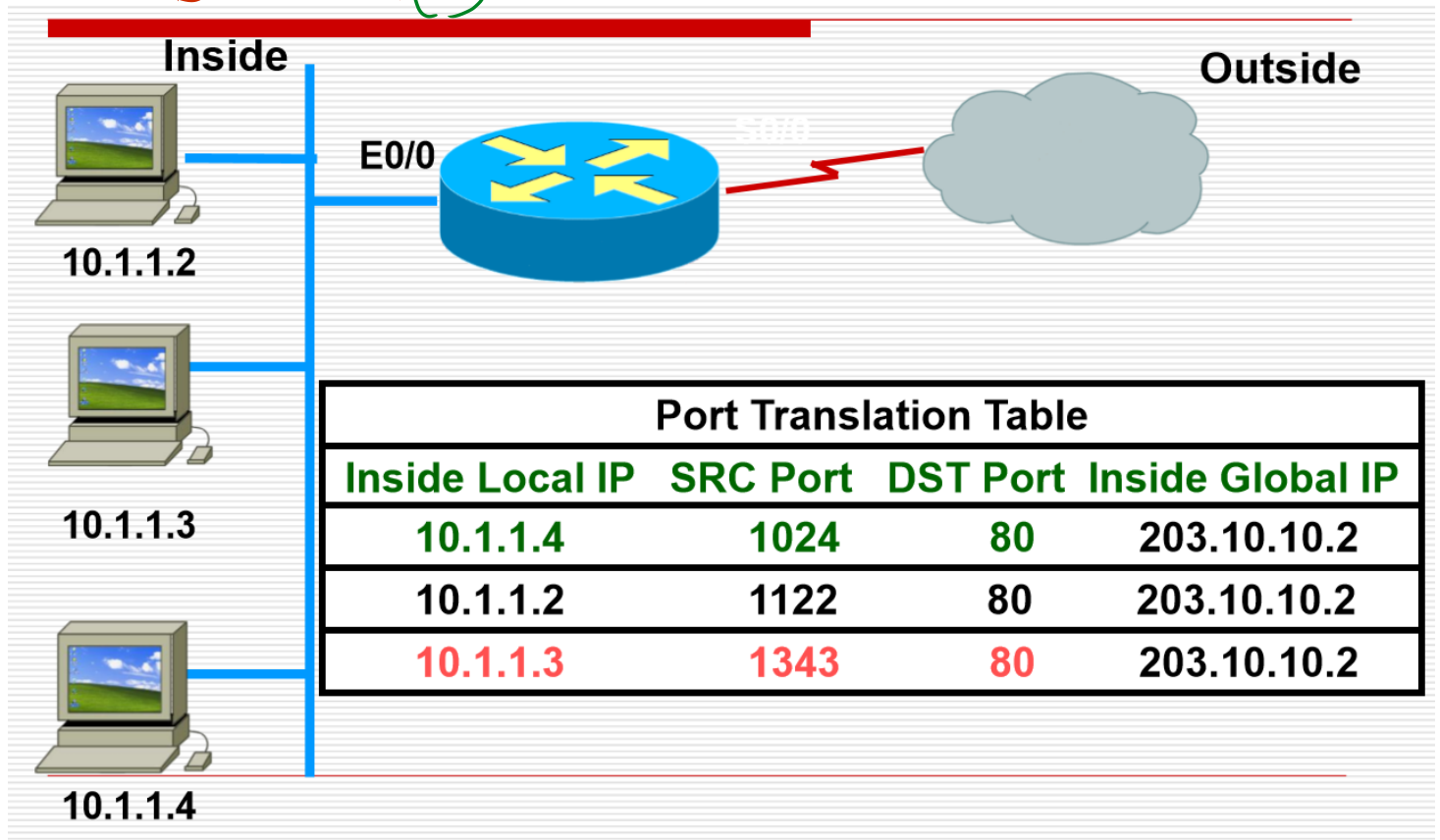
1. 指定一个地址池

4.7. NAT的优点和缺点

1. 优点：由于并非每个内部主机都需要同时进行外部访问，因此您可以使用少量的全局唯一地址池来服务相对大量的私有寻址主机。
2. 缺点：一一映射，并没有从根本上解决地址短缺的问题。
3. 也就是说，如果专用地址空间为/8，但公用地址为/24，则一次只能有254个主机可以访问Internet，主要内网不是同时有很多主机上网，就可以如上操作，进一步降低地址压力(类似并行和穿行的区别)

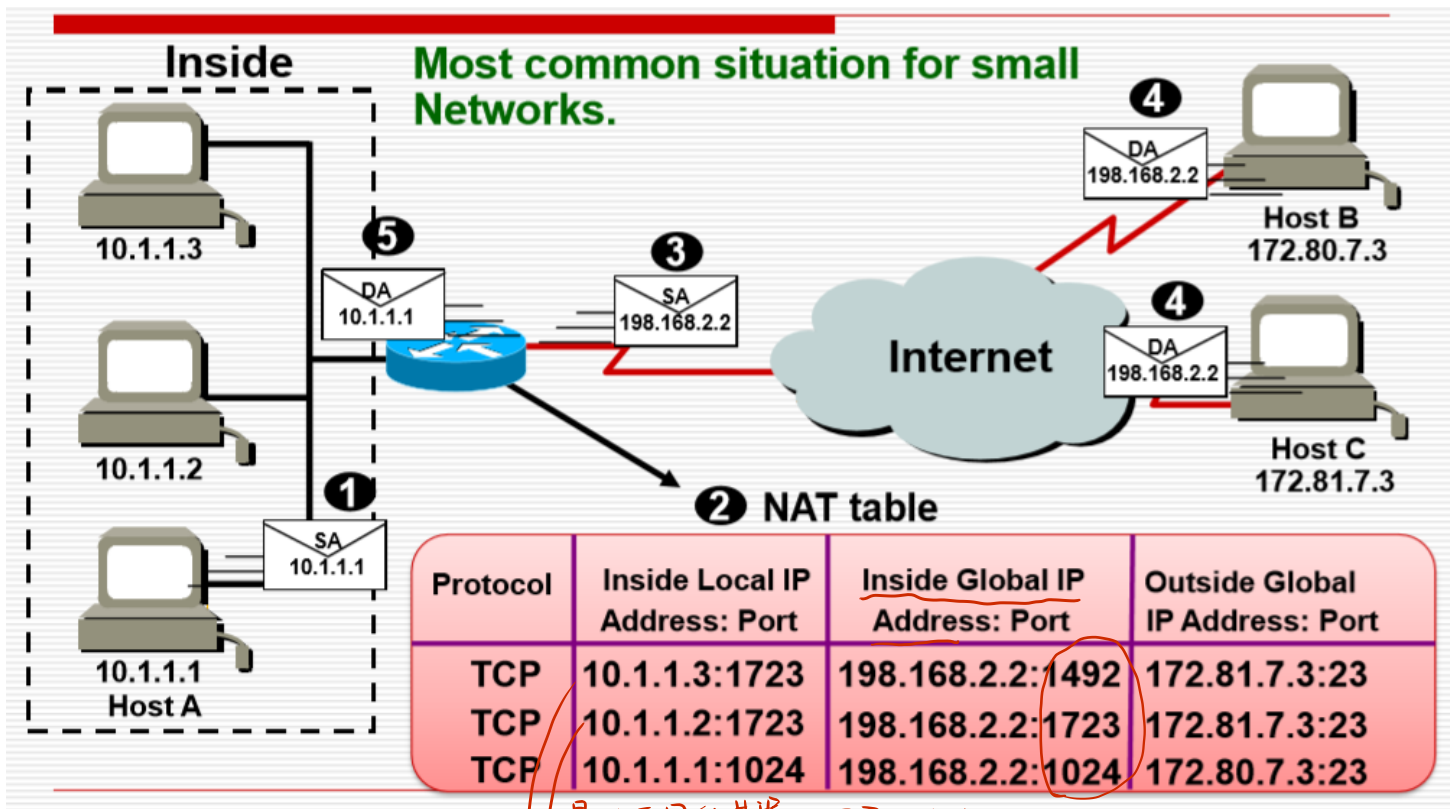
4.8. PAT的工作原理 (真正使用的类型)

通信的不是主机，是进程 => 做 socket 之间的映射



1. 第一个是IP
2. 可以有不同的端口
3. 同样的出口IP

4.9. PAT操作



1. 也就是我们对端口信息进行了调整

是3个不同的进程 => 不同socket

不同主机可以是相同端口。

看作是同一IP的不同进程

也因此是不同 port